

Postgres pgx Integration Coverage Push — REAL RUN evidence (§11.4.83)

Run date: 2026-06-23 **Run host:** nezha.local (Linux x86_64, go1.26.2, rootless podman 5.7.1 + podman-compose) **Driver host:** mistborn (macOS) — go1.26.2 but NO container runtime; tests were rsynced + run on nezha where go1.26 AND rootless podman both exist (mirrors the original docs/qa/20260623-postgres-integration/ staging at ~/helix_ota_itest).

Goal

Push the pgx/Postgres integration coverage (store 85.5% / rollout 83.1% from the prior real run) toward 100% by adding integration tests that exercise the genuinely-reachable pgx paths the existing contract tests do NOT drive.

Tests added (2 files, //go:build integration)

- `server/internal/store/postgres_coverage_integration_test.go` — `TestPostgresCoveragePaths_Integration`: boots real PG (port 55432, lock-serialized per §11.4.119) and drives:
 - **Project CRUD** (was 0.0%): `CreateProject` + unique-name conflict (23505→`ErrConflict`), `GetProject` found + `ErrNotFound`, `ListProjects` (ordered), `UpdateProject` found + `ErrNotFound` + rename-onto-existing-name conflict, `DeleteProject` found + already-gone `ErrNotFound`.
 - **Project access** (was 0.0%): `SetProjectAccess` insert + upsert (role change), `GetProjectAccess` found + `ErrNotFound`, `ListProjectMembers` (ordered), `RemoveProjectAccess` found + already-gone `ErrNotFound`, **FK ON DELETE CASCADE** proof (deleting the project removes its access rows).
 - **ListDevices** (was 0.0%): unfiltered, filter by `os_type` / `model` / `status`, offset-cursor paging (limit 2 → `page1+next`, `page2` no-cursor), no-match empty.
 - **NewPostgresRepositoryFromPool** (was 0.0%): the repo is constructed via this path.
- `server/internal/rollout/postgres_coverage_integration_test.go` — `TestPostgresStoreCoveragePaths_Integration`: boots real PG (port 55445, lock-serialized) and drives:
 - **NewPostgresStoreFromPool** (was 0.0%).
 - **Load** not-found → `engine.ErrNotFound` (the `pgx.ErrNoRows` branch).
 - **Save** with ZERO `PhaseStartedAt` (nil-timestamp branch) + 3-phase round-trip asserting zero round-trips as zero.

- **Save OVERWRITE**: a 2-phase plan + SET PhaseStartedAt — exercises the DELETE-then-reinsert phase-replacement path; Load reflects ONLY the new phases.

The two new tests do NOT modify the shared `contract_test.go` / `scenario_test.go` harnesses, so memory-repo coverage is unchanged — the delta is pure pgx.

REAL result — PASS, with measured coverage (from `integration_test_run.log`)

```
=== STORE coverage run ===
ok  internal/store      20.702s  coverage: 92.4% of statements
STORE_EXIT=0
=== ROLLOUT coverage run ===
ok  internal/rollout  13.971s  coverage: 87.3% of statements
ROLLOUT_EXIT=0
```

Package	BEFORE (prior real run)	AFTER (this run)	Δ
internal/store	85.5%	92.4%	+6.9pp
internal/rollout	83.1%	87.3%	+4.2pp

`go test -tags integration -count=1` → all integration + unit tests PASS, both package coverprofiles captured (`cov_store_after.out` / `cov_rollout_after.out` on nezha; func breakdown in `coverage_func_after.txt`).

pgx paths newly covered (func breakdown, from `coverage_func_after.txt`)

store/postgres.go: NewPostgresRepositoryFromPool 0.0%→100%, CreateProject 0.0%→100%, GetProject 0.0%→100%, GetProjectAccess 0.0%→100%, SetProjectAccess 0.0%→100%, ListProjects 0.0%→81.8%, UpdateProject 0.0%→88.9%, DeleteProject 0.0%→83.3%, ListProjectMembers 0.0%→81.8%, RemoveProjectAccess 0.0%→85.7%, ListDevices 0.0%→86.4%. rollout/postgres.go: NewPostgresStoreFromPool 0.0%→100%, Load 79.2%→higher, Save 73.3%→higher (overwrite + nil-timestamp branches now hit).

Genuinely-unreachable pgx lines left (honest §11.4.6)

The residual sub-100% lines are uniformly **driver-level fault branches** that cannot be triggered deterministically against a healthy booted DB without a fault-injection proxy: - `rows.Err()` / mid-iteration `rows.Scan` error returns in every `List*` / `Telemetry*` / `scan*` function (fire only on a connection drop / driver fault during a live query). - `pool.Query(...)` connection-failure return branches. - `Migrate` 66.7% — the `Exec` error branch = DDL failure, not triggerable on the valid

embedded schema. - jsonbOf 75% / nullTime 66.7% — the json.Marshal error path is **impossible**: Go never fails to marshal a map[string]string. This is defensive code by design.

These are NOT closed by contorting the test; closing them would require a mid-query connection-kill harness (a separate fault-injection work item), and the json.Marshal branch is structurally unreachable. Stated as a fact, not bluffed.

Cleanup (§11.4.14 — project-scoped)

- Boot via the digital.vasic.containers brick; teardown via the brick's own mgr.Shutdown (compose down scoped to postgres.compose.yml / postgres-rollout.compose.yml) on t.Cleanup — **never** an ancestor=filter (the prior incident).
- Pre-run container baseline captured: **63** containers (.containers_baseline_coverage.txt). Post-run count: **63** (exact match).
comm -13 baseline after = **empty** — no container from this run remains.
- Other projects' containers (helixcode, atmosphere, lava, ytdlp, llmsverifier, helixtranslate — incl. three pre-existing postgres:16-alpine from OTHER projects) were **NOT touched**. No helix_ota / port-55432 / port-55445 / deploy_postgres container is present.

Evidence files

- integration_test_run.log — combined store+rollout coverage run (both EXIT=0)
- coverage_func_after.txt — go tool cover -func full per-function breakdown (both pkgs)